Übungsblatt 6: Das Steuerwerk

Mikrocomputertechnik 1

Zielsetzung und Vorgehen

Sie leiten Steuersignale aus der Befehlswirkung her, unterscheiden Haupt- und ALU-Steuerung und bestimmen konkrete Folgen falscher Freigaben. Teil A ist ohne Simulator lösbar und übt Klausuraufgaben. Teil B vergleicht Vorhersagen mit Ripes.

Orientierung: Teil A etwa 60 Minuten und 50 Punkte zur Selbsteinschätzung, Teil B etwa 25 Minuten. Begründen Sie die Entscheidungen; eine Liste von Bitwerten allein genügt bei Erklärungsaufgaben nicht.

Vereinbartes Modell

Es gilt das Single-Cycle-Lehrmodell aus Vorlesung 5 und 6 mit der in Einheit 6 ergänzten `addi`-Unterstützung:

- Unterstützt: `add`, `sub`, `and`, `or`, `lw`, `sw`, `beq`, `addi`.
- Gültige Codierungen, 32-Bit-Daten, unkomprimierte Vier-Byte-Instruktionen, byteadressierter Speicher, Little Endian.
- Lesen kombinatorisch, Übernahme an der steigenden Flanke mit Freigabe. Alle genannten Speicheradressen sind gültig, beschreibbar und wortausgerichtet.
- `ALUSrc`: 0 wählt RD2, 1 den vorzeichenerweiterten Immediate.
- `MemtoReg`: 0 wählt das ALU-Ergebnis, 1 die Speicherlesedaten. X bedeutet eine nachweislich unerhebliche Auswahl.
- `ALUOp`: `00` erzwingt ADD, `01` SUB für `beq`, `10` wertet die gültigen Register-Funktionsfelder aus.
- `ALUControl`: `0000` AND, `0001` OR, `0010` ADD, `0110` SUB. Die Codes sind Kurskonventionen.
- PCSrc $=$ Branch $\wedge$ Zero. ImmGen liefert den fertigen Branch-Byteoffset ohne weiteren Shift.

Registernummern: `t0` $=$ `x5`, `t1` $=$ `x6`, `t2` $=$ `x7`, `s1` $=$ `x9`. Alle unabhängigen Fälle beginnen mit den jeweils angegebenen Anfangswerten.

Teil A: Papieraufgaben

Aufgabe 1: Hauptsteuerung herleiten (12 Punkte)

1. Füllen Sie die Quellen- und Freigabetabelle aus. Verwenden Sie X nur mit begründeter Unerheblichkeit. (6 P.)

Befehl	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch
`add`						
`lw`						
`sw`						
`beq`						
`addi`						

2. Geben Sie für jeden der fünf Befehle das zweibitige ALUOp an. Warum braucht `addi` dieselbe Kategorie wie `lw` und nicht dieselbe wie `add`? (3 P.)
3. Begründen Sie das X bei `sw` über Schreibport und Datenweg. Dürfte stattdessen `MemWrite` mit X bezeichnet werden? (2 P.)
4. Wie viele benannte Hauptsignale und wie viele Bits enthält die gesamte Hauptsteuerzeile? Gehört PCSrc als unabhängiger Ausgang dazu? (1 P.)

Ihre Lösung

Aufgabe 2: ALUOp, ALUControl und Bit 30 (10 Punkte)

Gegeben sind unabhängige Fälle mit `t1 = 7` und `t2 = 3`:

Fall	Befehl	Relevante Felder
A	`sub t0, t1, t2`	Opcode `0110011`, funct3 `000`, funct7 `0100000`
B	`or t0, t1, t2`	Opcode `0110011`, funct3 `110`, funct7 `0000000`
C	`addi t0, t1, -5`	Opcode `0010011`, funct3 `000`, Immediate `1111 1111 1011`

1. Bestimmen Sie ALUOp, ALUControl, die Quelle des zweiten ALU-Operanden und den neuen Inhalt von `t0` für alle drei Fälle. (6 P.)
2. Bit 30 ist in A und C gesetzt. Warum verlangt es nur in A eine Subtraktion? (2 P.)
3. Ein anderes Muster hat den Register-Opcode, funct3 `000` und funct7 `0000001`. Darf unser Decoder allein wegen Bit 30 gleich 0 auf `add` schließen? Begründen Sie die Antwort. (2 P.)

Ihre Lösung

Aufgabe 3: Boolesche Steuerbedingungen (10 Punkte)

R, L, S, B und I sind gegenseitig ausschließende Ein-Bit-Signale für gültig erkannte Registeroperationen, `lw`, `sw`, `beq` und `addi`.

1. Leiten Sie eine Boolesche Formel für RegWrite und eine für ALUSrc her. Verwenden Sie $\wedge$, $\vee$ und bei Bedarf den Überstrich zur Negation. (4 P.)
2. Zeichnen Sie RegWrite mit einem rechteckigen ODER-Gatter nach der verwendeten IEC-Darstellung. Beschriften Sie Ein- und Ausgang und die Bitbreiten. (2 P.)
3. Ergänzen Sie die folgenden Funktionen für jede gültige Befehlsgruppe. Warum sind beide zulässig? (3 P.)

$$\text{MemtoReg}_a = L, \qquad \text{MemtoReg}_b = L \vee S$$

Gruppe	R	L	S	B	I
MemtoReg a					
MemtoReg b					

4. Bedeutet X in der Steuertabelle einen hochohmigen Ausgang? Erklären Sie den Unterschied. (1 P.)

Ihre Lösung

Aufgabe 4: Fehlerfolgen exakt bestimmen (10 Punkte)

Zwei voneinander unabhängige, gedachte Decoderfehler im Lehrmodell:

Fall A: Speicherfreigabe bei Addition

```
add t0, t1, t2        # t1 = 0x2000; t2 = 4
```

Nur `MemWrite` ist falsch auf 1 gesetzt. Wort an `0x2004` vorher: 99. Alle übrigen Signale entsprechen einem korrekten `add`.

1. Welche Adresse und welche Schreibdaten liegen am Datenspeicher? (2 P.)
2. Welche Werte enthalten `t0` und das Wort an `0x2004` nach der Flanke? Geben Sie außerdem die vier Speicherbytes in aufsteigender Adressreihenfolge an. (2 P.)

Fall B: Registerfreigabe bei Store

```
sw t1, 12(s1)         # s1 = 0x2000; t1 = 37; x12 = 9
```

Hier sind `RegWrite = 1` und `MemtoReg = 0` gesetzt. Alle anderen Signale entsprechen einem korrekten Store. Die Bits 11:7 der Instruktion enthalten die unteren fünf Offsetbits, also `01100`; der Registeradressport ist fest mit diesen Positionen verbunden.

3. Bestimmen Sie den Speicher-Schreibort und den geschriebenen Wert. Welches Register wird zusätzlich beschrieben und mit welchem Wert? (3 P.)
4. Warum sind die Folgen in beiden Fällen nicht „zufällige Zellen mit Müll“? Warum setzt das Don’t Care bei MemtoReg eine korrekte Registerfreigabe voraus? (3 P.)

Ihre Lösung

Aufgabe 5: Branch-Freigabe und Entscheidung (8 Punkte)

Der aktuelle Befehl liegt bei `PC = 0x100`. Sein vollständiger Branch-Byteoffset zum Label `ziel` beträgt $+12$.

1. Ergänzen Sie die Tabelle. C ist eine unabhängige Registeroperation an derselben PC-Adresse. (4 P.)

Fall	Befehl und Operanden	Branch	Zero	PCSrc	Neuer PC
A	`beq t0, t1, ziel`; 5 und 5				
B	`beq t0, t1, ziel`; 5 und 6				
C	`sub t0, t1, t2`; 5 und 5				

2. Welche ALUOp- und ALUControl-Werte gelten in A und B? Welche in C? (2 P.)
3. Warum bleibt Branch auch beim nicht genommenen Sprung 1? Warum wird der Byteoffset hier nicht nochmals verschoben? (2 P.)

Ihre Lösung

Teil B: Beobachtung in Ripes

Aufgabe 6: Ein Programm, mehrere Steueraufgaben

Verwenden Sie `beispiele/vl6_mux_beobachter.s`. Wählen Sie einen RV32-Single-Cycle-Prozessor, deaktivieren Sie C und notieren Sie Version und Prozessorbezeichnung. Lesen Sie die tatsächliche Adresse von `slot` als B ab.

Die Initialisierung `la s1, slot` kann mehrere Maschineninstruktionen erzeugen. Beginnen Sie bei `Beobachtung`. Halten Sie vor den Exit-Anweisungen bei `Pruefpunkt` an; die Simulator-Dienstkonvention ist für die Aufgaben nicht erforderlich.

Vor jedem Clock-Schritt: PC und Befehl, Operanden, ausgewählte Registerschreibquelle und Register-/Speicherfreigaben protokollieren. Nach dem Schritt: gespeicherte Register, Speicherwort und PC prüfen. Die dann sichtbaren Leitungen können schon zur nächsten Instruktion gehören.

Vorhersage und Protokoll

1. Ergänzen Sie zunächst die erwarteten Zustandsänderungen. (ohne Punkte)

Befehl	Nach Clock erwartet	Nach Clock beobachtet
`addi t0, x0, 5`		
`add t1, t0, t0`		
`sw t1, 0(s1)`		
`lw t2, 0(s1)`		
erster `beq`		
`addi t3, x0, 1`		
zweiter `beq`		

2. Welche konkrete Register-Rückschreibquelle zeigt Ripes beim Store und beim Branch? Warum kann ein angezeigter Auswahlwert mit X in der Papiertabelle vereinbar sein?
3. Welches Bauteil berechnet den Branch-Vergleich, welches das Ziel? Vergleichen Sie die Funktionen mit dem Lehrmodell, ohne gleiche interne Codes vorauszusetzen.
4. Dokumentieren Sie die Register- und Speicherwerte am Prüfpunkt und begründen Sie, warum der Markerwert 99 nicht ausgeführt wird.